

➤ **What is Natural language processing (NLP) ?**

- **Natural language processing (NLP), as a subfield of Artificial Intelligence, applies computer techniques in linguistic research, thus implying usage of algorithms, data structures, formal models of representation and reasoning, as well as specific AI techniques (especially search and knowledge representation methods).**
- **NLP ultimately creates a technology, namely an ensemble of processes, methods, techniques, that creates and implements ways of carrying out various tasks referring to natural language.**

➤ **Natural language engineering deals with the implementation of large-scale systems.**

➤ **Applications of NLP**

- **Applications based on text:**
 - ✓ document classification
 - ✓ information retrieval
 - ✓ information extraction
 - ✓ text understanding
 - ✓ knowledge acquisition
 - ✓ machine translation and machine aided translation
- **Applications based on dialogue:**
 - ✓ human-computer interaction
 - ✓ learning systems
 - ✓ question answering systems

and others.

N.B. Note that:

- **Information retrieval has expanded and tends to become an autonomous field (that interacts with NLP).**
- **Speech processing also tends to represent an autonomous field.**
- **It is important to distinguish between natural language understanding and speech recognition. Speech recognition deals only with identifying spoken words that originate from a given signal. It does not deal with message UNDERSTANDING, which remains a task belonging to the field of NLP.**

LEVELS OF NATURAL LANGUAGE PROCESSING

➤ The Morphological level

- The word is studied from the grammatical perspective, namely with respect to the variation of its form (inflexion). A change in the form of a word (usually by adding a suffix) indicates a change in its grammatical function.**
- At morphological level we assign a part of speech (noun, verb, adjective etc.) to each word. This is done automatically by using a POS-tagger.**
- The process of assigning part of speech tags (noun, verb, adjective etc.) to tokens (words) is called POS-tagging.**

(See the corresponding slide referring to data preprocessing).

➤ **The Syntactic level** (Syntactical analysis $\equiv$ parsing)

- The analysis at this level is organized as taking place between the word – as minimal entity – and its possible combinations: phrases, sentences – as maximal entities.

N.B. Here “phrase” is not the Romanian “fraza”, but can signify any group of words. **Definition:** a phrase is an expression consisting of one or more words forming a grammatical constituent of a sentence.

- The syntactic level of analysis determines the structural role of each word within a sentence.
- The process of automatically determining the structure of a sentence is called parsing. The corresponding computer program is a parser.

- The type of parsing (syntactical analysis) that we perform depends on the type of grammar that is used.
- Example: A generative grammar (Noam Chomsky 1957, 1984) leads to parsing based on constituents; a dependency grammar (Hudson 1984) leads to dependency parsing. (See the lectures on syntactical analysis – coming up!)
- N.B. Here a grammar is considered an instrument for knowledge representation concerning a natural language.

- ❖ To examine how the syntactic structure of a sentence can be computed, you must consider two things: the grammar, which is a formal specification of the structures allowable in the language, and the parsing technique, which is the method of analyzing a sentence to determine its structure according to the given grammar.

➤ **The Semantic level (Computational semantics)**

- **The semantic level refers to:**
 - **the meaning of words and sentences;**
 - **the way in which word meanings combine in order to form the meaning of an entire sentence.**
- **Computational semantics performs a context-independent study of meaning. Namely, we are interested in the meaning of a sentence independently of the context in which it occurs.**
- **One problem that arises when performing semantic analysis is that of word ambiguity – requiring word sense disambiguation (WSD).**

➤ **The Pragmatic Level**

- **Pragmatics deals with usage of language in context.**
- **In NLP the pragmatic level studies:**
 - **usage of sentences in various contexts;**
 - **the way in which context influences the interpretation of a sentence.**
- **Here the context plays an essential role.**
- **Pragmatic analysis becomes very difficult in the presence of various phenomena that are typical to discourse in natural language. See, for instance, the reference relation, as in:**

John sustains he doesn't believe this.

(here reference occurs within the same sentence)

and

John thinks he found his hat.

(here reference can involve different sentences; “his” could refer to John, occurring in the same sentence, or to someone else, who has been mentioned in a different sentence).

- ❑ **This introductory course will guide you to work at the morphological, syntactic and semantic levels of NLP.**

Representing the structure of a sentence

- The most common way of representing how a sentence is broken into its major subparts, and how those subparts are broken up in turn, is a **tree**.
- In order to represent such trees, we introduce the concept of **syntactic phrase** (*grup sintactic*).

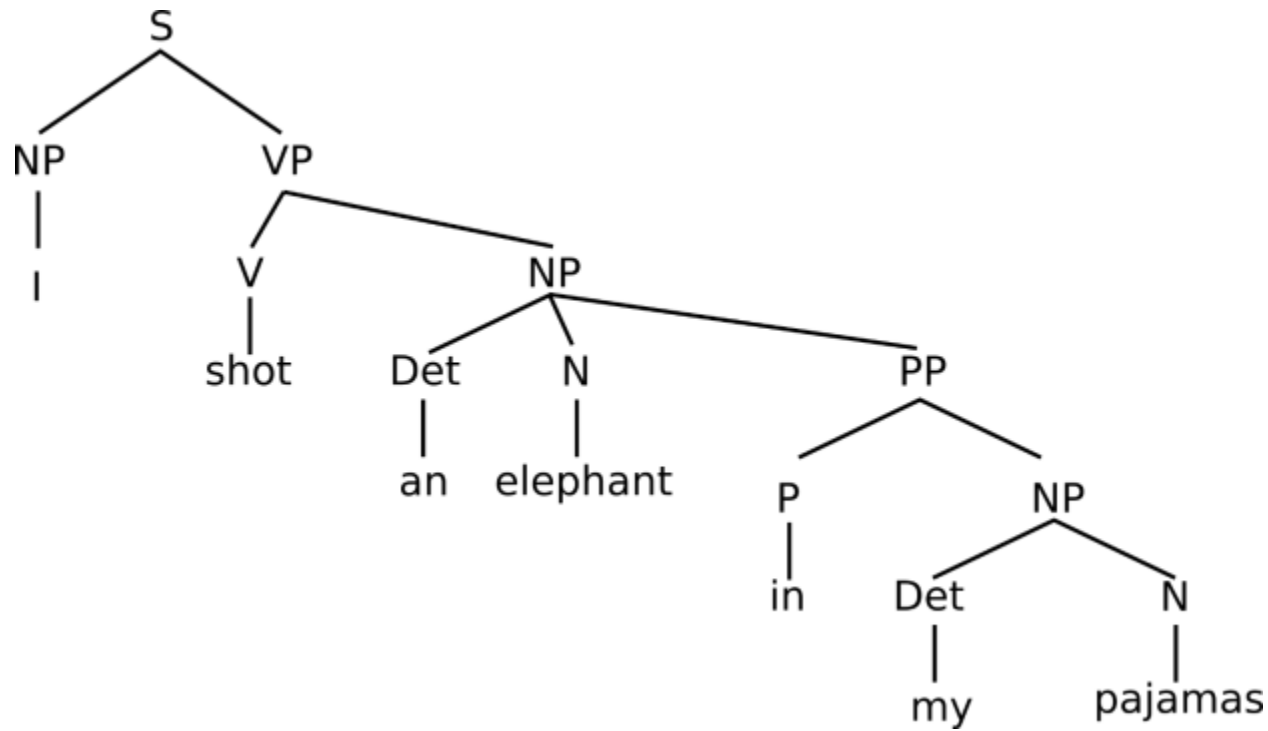
Syntactic phrase

- **A syntactic phrase is a component of the structure of a sentence organized around a head that can be a verb, a noun, adjective, adverb or preposition. The syntactical-semantic cohesion of the group of words representing the syntactic phrase is ensured by grammatical constraints (of type case, topic, agreement) and by the semantic roles imposed by the head on the determinants.**
- **According to the morphological class of the head, we distinguish:**
 - **verb phrase – notation VP (the head is a verb)**
 - **noun phrase – notation NP (the head is a noun)**
 - **adjectival phrase – notation ADJP (the head is an adjective)**
 - **adverbial phrase – notation ADVP (the head is an adverb)**
 - **prepositional phrase – notation PP (the head is a preposition)**
- **S is the syntactic phrase that stands for the entire sentence.**

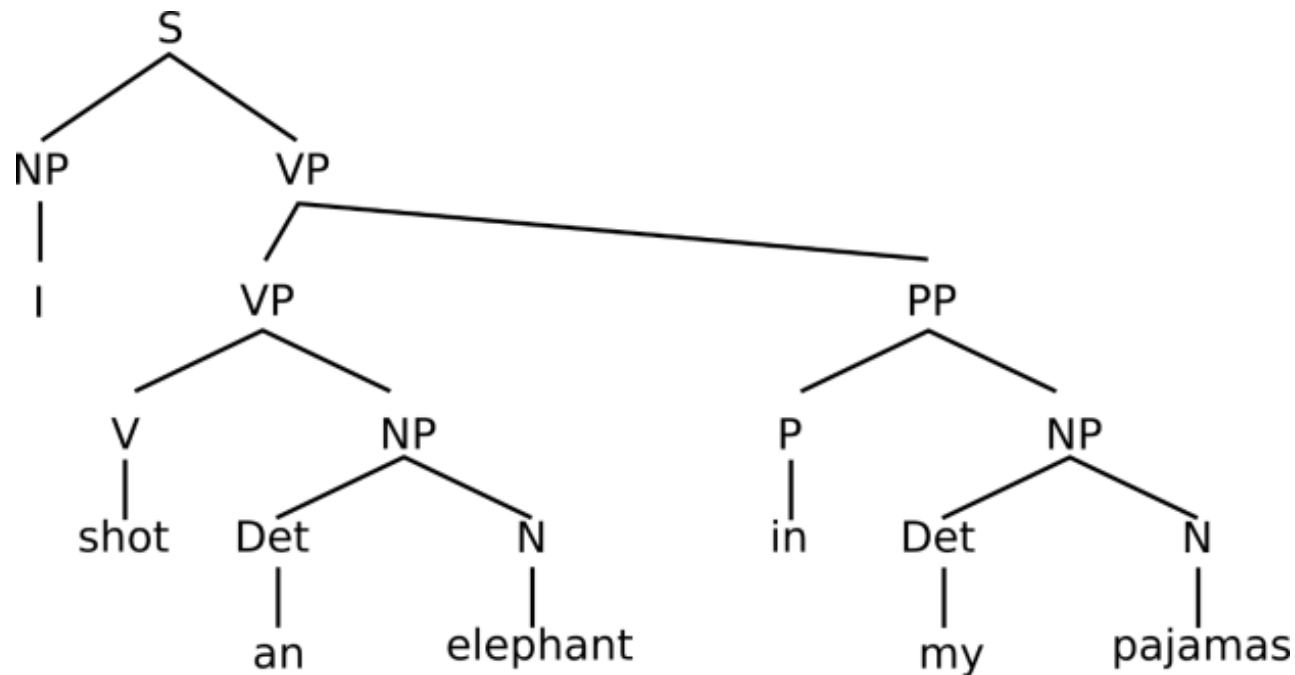
- Syntactic phrases can be used to represent the syntactic structure of a sentence, as in the following example:

“While hunting in Africa, I shot an elephant in my pajamas. How an elephant got into my pajamas I’ll never know”.

Parse tree no. 1



Parse tree no. 2



Parse tree no.1 corresponds to the correct sense, which can only be disclosed within the context. Here the elephant is inside the pajamas.

This is an example of ambiguity. The existence of two parse trees denotes structural ambiguity that leads to semantic ambiguity as well.

Ambiguity in natural language processing

- Ambiguity is a major problem in natural language processing.
- There are various kinds of ambiguity an NLP system has to deal with:
- ✓ structural ambiguity - occurs when a parser detects more than one possible structure corresponding to an input sentence (as in the previous example)
- ✓ lexical ambiguity – occurs when a word can have multiple parts of speech as in:
Rice flies like sand.
- ✓ semantic ambiguity – occurs in the case of polysemous words (words having multiple senses) as in the following Romanian sentences:

Kilogramul de rosii face 10 lei.

Ion face arhitectura.

Andrei face probleme.

- Note that one type of ambiguity can lead to another. (For instance, lexical ambiguity can lead to structural ambiguity that can lead to semantic ambiguity etc.)

- **Before conducting any kind of text analysis, there is a data preprocessing step (preprocesarea datelor) that must be performed!**

Data preprocessing

- Tokenization - splitting the text into tokens (minimal meaningful units); we tokenize a *sentence* into *words* or an *article* into *sentences*
- POS - tagging – assigning a part of speech to each word (noun, adjective etc.)
- Stemming - the process of reducing a word to its stem (ex.: *walking* is reduced to *walk*)
- Lemmatization - similar to stemming except it operates also by including the knowledge about the word's context (ex.: *good* and *better*)
- NER - Named Entity Recognition – labels a sequence of words that represents the names of things (ex.: a company, a street, a person's name)
- Parsing - a parser analyzes the grammar of a text in order to extract or determine its structure

The difference between stemming and lemmatization

The aim of both processes is the same: reducing the inflectional forms of each word into a common base or root. However, these two methods are not exactly the same.

- Stemming algorithms work by cutting off the end or the beginning of the word, taking into account a list of common prefixes and suffixes that can be found in an inflected word. This indiscriminate cutting can be successful in some occasions, but not always. Example: form – studies (the verb “study”); suffix – es; stem – studi
- Lemmatization takes into consideration the morphological analysis of the words. To do so, it is necessary to have detailed dictionaries which the algorithm can look through in order to link the form back to its lemma. Example: form - studies; morphological information - third person, singular number, present tense of the verb *study*; lemma – study

Note:

1. A lemma is the base form of all its inflectional forms (toate “formele flexionare” ale cuvântului), whereas a stem isn’t.
2. Regular dictionaries are lists of lemmas, not stems. (Lema este forma din dicționar).

NLP tools and libraries

- **Core NLP (Stanford CoreNLP):** contains tools for tagging, parsing, analyzing text; is written in JAVA; supports other languages besides English (Arabic, French, Chinese, German, Spanish etc.)
- **spaCy:** created for industrial strength NLP; provides most mentioned functionalities (POS-tagging, tokenization etc.); supports various languages (German, Spanish, Portuguese, French, Italian, Dutch etc.); the 2017 version has 92,6% accuracy
- **gensim:** for semantic analysis (text mining); the author is a Czech researcher
- **NLTK – Natural Language Toolkit – “the mother of all NLP libraries”:** developed since 2001; well maintained and modular; comes with big amount of data, corpora and trained modules;
book:

nltk.org/book

NLTK

Tokenization

- The process of breaking up text into smaller pieces (tokens)
- Token: can be a word or a sentence
- `import nltk` and see demo file

Stop words

- they are removed from the text
- usually they are language specific
- they are words like “then”, “before”, “between”, the” etc.
- these are words that are filtered out before processing natural text
- removing them is part of data processing (a stop word can be dominant in a text and must be removed from the text because it doesn't import anything to what the text is about)
- with NLTK you import stop words corresponding to the language your text is written in (NLTK provides lists of stop words)
- punctuation must also be removed
- see demo file

POS-tagging

- the process of assigning part of speech tags (noun, verb, adjective etc.) to tokens (words)

```
>>> import nltk
>>> from nltk import pos_tag
...
>>> text = nltk.word_tokenize("And now for something
completely different")
>>> nltk.pos_tag(text)
[('And', 'CC'), ('now', 'RB'), ('for', 'IN'),
('something', 'NN'), ('completely', 'RB'), ('different',
'JJ')]
```

where:

CC designates a coordinating conjunction

RB designates an adverb

IN designates a preposition

NN designates a noun

JJ designates an adjective

- NLTK provides documentation for each tag, which can be queried using the tag
`nltk.help.upenn_tagset('RB')`

Named Entity Recognition

- **NER is used to recognize entities in a text - like people, organisations, places etc.**
- **One must import named entity chunk and also a sentence tokenizer:**
from nltk import ne_chunk, sent_tokenize,
word_tokenize
where:
sent_tokenize(text) - returns a sentence-tokenized copy of text,
using NLTK's recommended sentence tokenizer
word_tokenize(text) - returns a tokenized copy of text, using
NLTK's recommended word tokenizer
ne_chunk(tagged_tokens) - Use NLTK's currently
recommended named entity chunker to chunk the given list of
tagged tokens
- **See demo file**

TF - IDF

TF: term frequency

IDF: Inverse Document Frequency

TF = (number of times the term appears in a document) / (total number of terms in the document)

IDF = $\log(\text{total number of documents}) / (\text{number of documents with the word in it})$

TFIDF = TF * IDF

Example: We look at the Brown corpus, coming with NLTK, and we extract the key words

- **See the program TF-IDF.py on the slide**
“Example TFIDF.docx” included in the folder “Demo file”
- **The code there uses Porter stemmer from NLTK**
- **For the code to run smoothly you need to install the Python library called numpy**
- **The program initializes a list of stop words (for English, taken from NLTK)**

POS-Tagging and Grammars

POS-tagging forms the basis of any further syntactic analyses.

Exploring the In-Built Tagger

```
import nltk
# Create a variable called simpleSentence and assign a
# string to it
simpleSentence = "Bangalore is the capital of Karnataka."
# Use the nltk built-in tokenizer function called
# "word_tokenize"
wordsInSentence = nltk.word_tokenize(simpleSentence)
# It breaks a given sentence into words and returns a Python
# list data type; we assign it to the variable wordsInSentence
# Display the given data structure (a Python list)
print(wordsInSentence)
['Bangalore', 'is', 'the', 'capital', 'of', 'Karnataka', '.']
# Invoke the nltk built-in tagger, called pos_tag, which takes
# the list of words and identifies the part of speech of each
# word in wordsInSentence
partsOfSpeechTags = nltk.pos_tag(wordsInSentence)
# We see a list of tuples where each tuple has the tokenized
# word and the POS identifier
partsOfSpeechTags = nltk.pos_tag(wordsInSentence)
# invoke the print function
print(partsOfSpeechTags)
[('Bangalore', 'NNP'), ('is', 'VBZ'), ('the', 'DT'), ('capital', 'NN'), ('of', 'IN'), ('Karnataka', 'NNP'), ('.', '.')]

```

Learning to Write Your Own Grammar

```
# import the string module into your own program
import string
# import the generate function from nltk.parse.generate
# module
from nltk.parse.generate import generate
# the grammar can contain some production rules; the
# starting symbol is ROOT
productions = [
    "ROOT -> WORD",
    "WORD -> NUMBER LETTER",
    "WORD -> LETTER NUMBER",
]
# all production rules are converted to a string
grammarString = "\n".join(productions)
# a new grammar object
# we use the CFG.fromstring method which takes the
# grammarString variable that was just created
grammar=nltk.CFG.fromstring (grammarString)
# print the grammar
print(grammar)
```


Writing a Recursive CFG

(CFG = Context-Free Grammar)

```
productions = [  
    "ROOT->WORD"  
    "WORD->' '"  
]  
  
# Retrieve the list of decimal digits as a list in the alphabets  
# variable  
alphabets = list(string.digits)  
# Using the digits from 0 to 9 we add more productions to  
# our list  
for alphabet in alphabets:  
    productions.append("WORD->' {w}' WORD' {w}'".  
        .format(w = alphabet))  
# concatenate all the rules  
grammarString = "\n".join(productions)  
# create a new grammar object  
grammar = nltk.CFG.fromstring(grammarString)  
print(grammar)
```